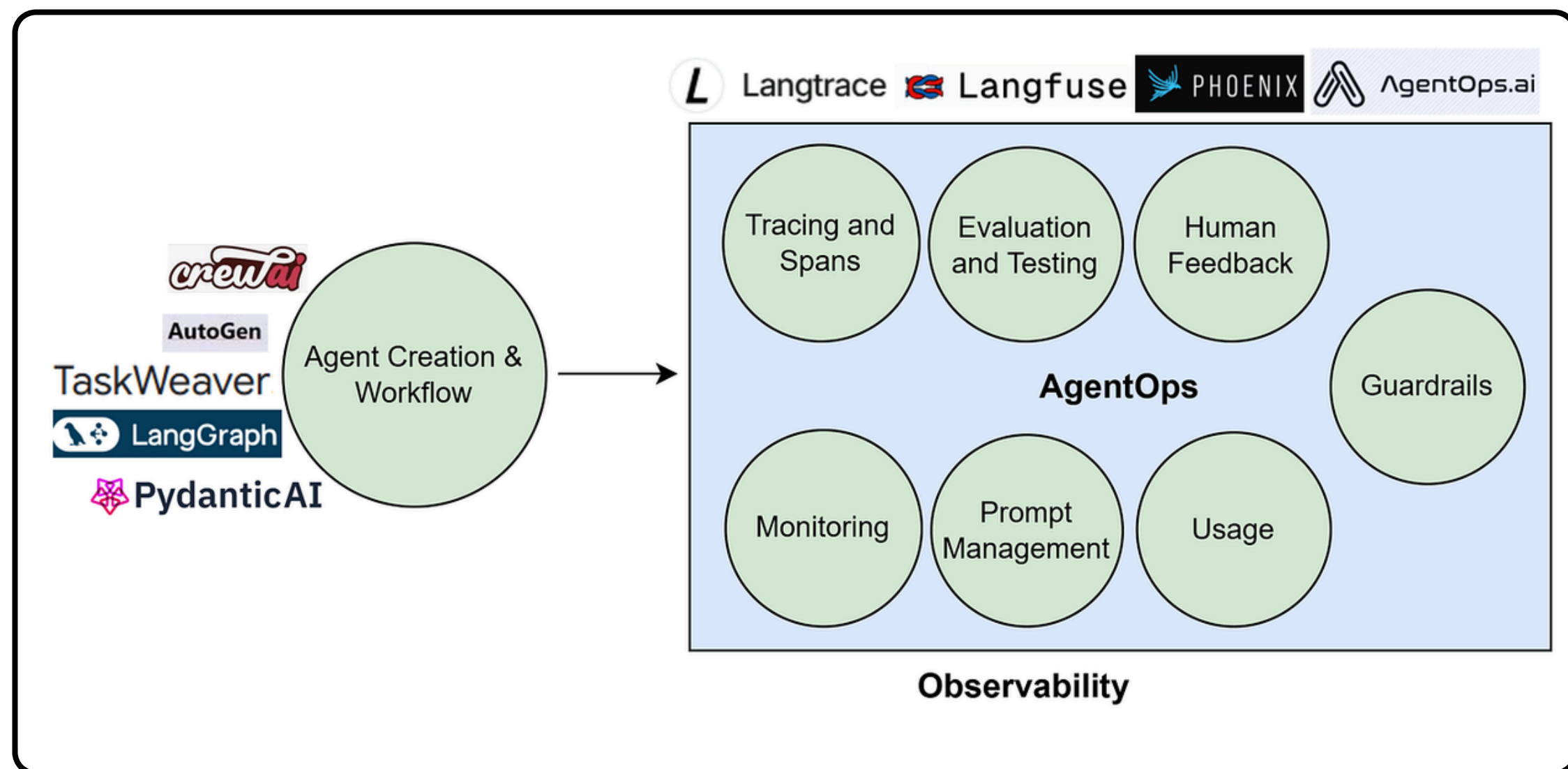
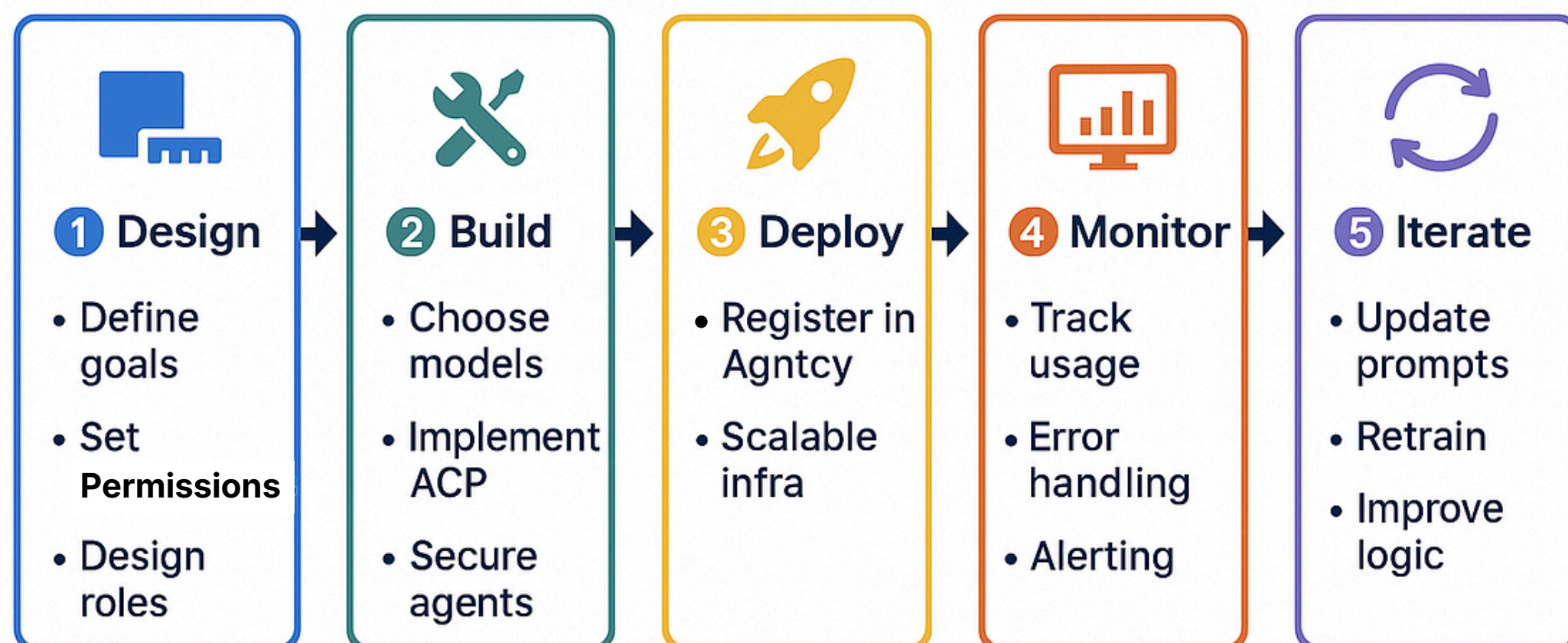


AgentOps Observability



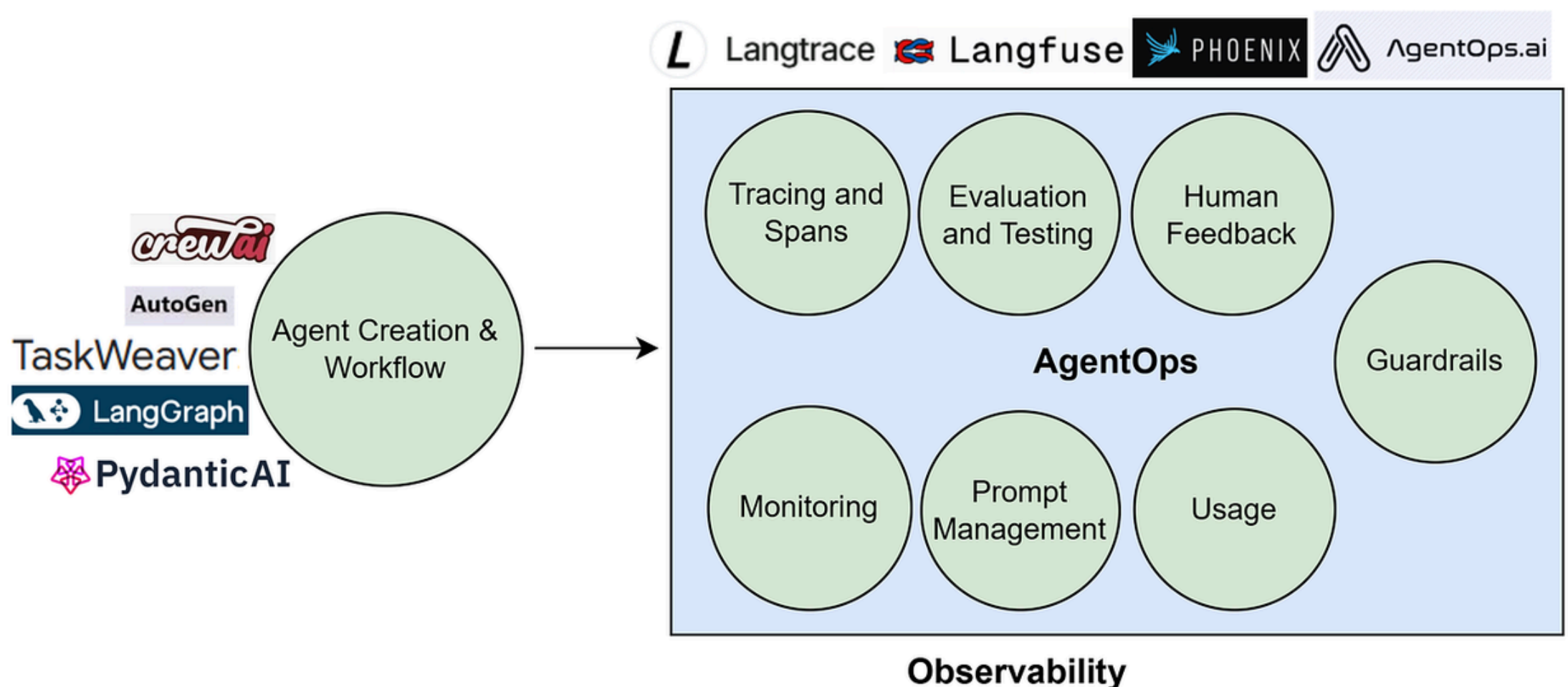
AgentOps Lifecycle



AgentOps refers to the development and operational practices used to **build, evaluate, deploy, and monitor AI agents**. In this context, observability involves instrumenting agent workflows to capture telemetry such as logs, metrics, and traces across all stages of execution, including LLM calls, tool interactions, and sub-agent actions.

This enables a complete view of agent behavior, supports reliable multi-step task execution, and helps identify errors or biases. The AgentOps framework models each LLM or tool interaction as an event and offers dashboards for session replay and performance insights.

Industry platforms, including Azure AI and IBM, similarly provide step-level tracing capabilities for agent workflows. Collected telemetry spans LLM and API calls, decision points, and key metrics such as token usage and memory consumption



Distributed Tracing Concepts & Instrumentation

Distributed tracing captures the path of a request (or agent “run”) as a tree of spans across services. Each span represents an operation (e.g. calling an LLM or API) and carries attributes (metadata) and timing. Spans are linked in a trace, with a unique trace ID propagated in headers (HTTP or custom) so that end-to-end context is maintained across processes. OpenTelemetry (OTel) is the CNCF standard for traces (and metrics/logs). It defines APIs/SDKs for instrumentation and a collector for pipelines.

Instrumenting code means **creating** and **reporting spans**. For example, in Python one creates a tracer and starts spans with ***tracer.start_as_current_span("name"): blocks***. The OpenTelemetry Python example (Flask route) shows manual instrumentation:

```
with tracer.start_as_current_span(
    "server_request",
    context=extract(request.headers),
    kind=SpanKind.SERVER,
    attributes=collect_request_attributes(request.environ),
):
```

OTel's Python SDK setup (shown below) adds a TracerProvider, processors, and exporters (e.g. console or OTLP). The official docs show using BatchSpanProcessor and an exporter:

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor, ConsoleSpanExporter

# Create provider and processor/exporter
trace.set_tracer_provider(TracerProvider())
tracer = trace.get_tracer(__name__)
span_processor = BatchSpanProcessor(ConsoleSpanExporter())
trace.get_tracer_provider().add_span_processor(span_processor)

# Create and end a span
with tracer.start_as_current_span("span-name") as span:
    span.set_attribute("example.key", "value")
```

OpenTelemetry provides automatic instrumentation libraries (e.g. opentelemetry-auto-instrumentation in Python) that monkey-patch frameworks (Flask, requests, etc.) to generate spans with minimal code changes. One can launch apps via opentelemetry-bootstrap or --instrument flags to auto-instrument.

For Node.js, the OTel Node SDK allows auto-instrumentation. A typical setup uses NodeSDK with auto-instrumentation plugins:

```
import { NodeSDK } from '@opentelemetry/sdk-node';
import { ConsoleSpanExporter } from '@opentelemetry/sdk-trace-node';
import { PeriodicExportingMetricReader, ConsoleMetricExporter } from '@opentelemetry/sdk-metrics';
import { getNodeAutoInstrumentations } from '@opentelemetry/auto-instrumentations-node';

const sdk = new NodeSDK({
  traceExporter: new ConsoleSpanExporter(),
  metricReader: new PeriodicExportingMetricReader({
    exporter: new ConsoleMetricExporter(),
  }),
  instrumentations: [getNodeAutoInstrumentations()],
});
sdk.start();
```

This auto-instruments Express HTTP requests, and the example shows new spans printed by the ConsoleSpanExporter. The Node example uses `--import instrumentation.ts` when running to preload instrumentation before the app code.

In Go, the OpenTelemetry Go SDK similarly offers trace packages (e.g. go.opentelemetry.io/otel), and standard instrumentations for HTTP clients/servers.

For brevity we omit details, but typical usage is analogous: create a TracerProvider, use `otel.Tracer("name").Start(ctx, "operation")`, and propagate context manually or via middleware.

A key concept is context propagation: the trace ID is propagated across threads/processes via carriers (HTTP headers, gRPC metadata, etc.), using W3C Trace Context format. All major languages support the W3C traceparent header. OpenTelemetry ensures spans created in one service are correlated with a parent span from upstream. If propagation is misconfigured, traces will appear fragmented.

Sampling and Overhead

Collecting every trace can be expensive. OpenTelemetry allows sampling: deciding which traces to record. Basic samplers include AlwaysOn, AlwaysOff, and probabilistic (TraceIdRatioBased). The OTel sampling guide notes that for high-volume systems, a small sampling rate (e.g. 1%) often suffices to represent overall behavior.

It says: "For high-volume systems, it is quite common for a sampling rate of 1% or lower to very accurately represent the other 99% of data". Samplers can also be configured by service or error status (e.g. sample all error traces). Sampling greatly reduces cost and storage, but risks missing rare events.

Best practice is to sample selectively (e.g. keep all errors, or use a parent-based sampler that samples differently per operation). OpenTelemetry provides parent-based and tail-based samplers in collector.

Spans and Attributes

Each span can carry attributes (key-value tags) describing the operation (e.g. SQL query, HTTP URL, tool name). Use semantic conventions (OTel's semantic conventions for resource names). Attach enough context (user IDs, resource names, error codes) to diagnose issues, but avoid sensitive data (see Security below).

Spans can have events (like logs or exceptions) attached. Traces can include distributed call graphs; for example, Google's ADK AgentOps trace example shows spans for each agent call and tool interaction.

Monitoring Metrics

Beyond traces, metrics provide numerical observability. Common metric types (from Prometheus documentation) are: Counters (monotonic counts, e.g. total requests), Gauges (values that go up/down, e.g. concurrent sessions), Histograms (bucketed observations, e.g. latency), and Summaries (sliding-window quantiles). For agent monitoring, metrics might include token usage count, API call rates, CPU/memory usage, error rates, etc. Prometheus is the de-facto monitoring system. Instrumentation is done via client libraries (for Go, Java, Python, Node, etc.). Each instrumented app exposes an HTTP /metrics endpoint in Prometheus text format.

Prometheus scrapes these to collect data. Official docs list languages with client libs (e.g. for Python, see lines 104–108). Example: in Python, one might use `prometheus_client` to define a counter:

```
from prometheus_client import Counter, start_http_server
requests = Counter('agent_requests_total', 'Total agent requests')
start_http_server(8000) # expose /metrics
requests.inc()
```

See Prometheus Client Libraries doc. Alternatively, one can use OpenTelemetry Metrics API and a Prometheus exporter (converting OTel metrics to Prom metrics).

Exporters and integrations: If a system isn't natively instrumented, use a Prometheus exporter. Prometheus docs list hundreds (database exporters, JMX, HAProxy, etc.). The OpenTelemetry Prometheus Exporter is common: it runs as a pull endpoint. OTel spec requires that a Prometheus exporter use client libraries to serve metrics and conform to text format 0.0.4. In practice, the OTel Collector's prometheus exporter or the OTel SDK Prometheus exporter are used. The collector approach decouples apps from Prometheus by collecting OTel metrics and exposing them via `/metrics`.

Prometheus Agent Mode: An important pattern is running Prometheus itself as a sidecar agent. In "Agent mode" (as of Prometheus 2.40+), the binary skips local storage/alerts and simply scrapes apps and remote-writes to a central store. The docs explain: "Agent mode optimizes Prometheus for the remote write use case. It disables querying, alerting, and local storage...". The diagram below shows a Prometheus agent scraping metrics and sending to a remote write endpoint (e.g. a central Prometheus or Cortex).